

Дискретная математика

Конечные автоматы

Константин Чухарев

Осень 2026

Языки

Computer science — это не про компьютеры, как астрономия — не про телескопы.

— Эдсгер Дейкстра

Алфавит, слово, язык

Программа читает строку и решает, подходит она под формат или нет.

Прежде чем строить автомат, договоримся о точном языке для строк.

Определение 1

- **Алфавит** — конечное множество символов Σ .
- **Слово** — конечная последовательность символов.
- **Язык** — подмножество $L \subseteq \Sigma^*$.

Пример

- Двоичный алфавит $\{0, 1\}$.
- Слово 1001, пустое слово ϵ .
- Язык слов, заканчивающихся на 01.

Распознавание

Распознать язык — значит решить, принадлежит ли слово w множеству L .

Чтобы распознавать язык, его надо как-то задать — *конечным описанием*: Конечных описаний счётное множество ($\aleph_0$), а языков — несчётное ($2^{\aleph_0} > \aleph_0$).

Почти любой язык конечной записью не описать — а значит, и распознать нельзя.

Мы изучаем класс языков, которые описать *можно*.

Конкатенация

Определение 2

- **Конкатенация:** $10 \cdot 01 = 1001$.
- **Степень:** $u^0 = \varepsilon$, $u^{n+1} = u^n u$.
- **Для языков:** $L_1 L_2 = \{uv \mid u \in L_1, v \in L_2\}$.

Пример

$$\{a, ab\} \cdot \{c\} = \{ac, abc\}.$$

Слова — это то, что машина читает.

Конечная память — номер состояния.

Детерминированные конечные автоматы

Определение ДКА

Определение 3 (ДКА)

Пятёрка $M = (Q, \Sigma, \delta, q_0, F)$:

- Q — конечное множество состояний.
- Σ — алфавит.
- $\delta : Q \times \Sigma \rightarrow Q$ всюду определена.
- $q_0 \in Q$ — начальное состояние.
- $F \subseteq Q$ — принимающие.

Детерминизм: следующее состояние известно заранее.

Вся память — номер текущего состояния.

Пример

Определение ДКА [2]

Язык с чётным числом единиц распознаётся двумя состояниями. Символ 0 состояние не меняет, символ 1 переключает.

Язык автомата

Определение 4 (Расширенная функция переходов)

$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ доопределяет δ на целые слова и задаётся рекурсивно:

$$\hat{\delta}(q, \varepsilon) = q.$$

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a) \text{ для } a \in \Sigma, w \in \Sigma^*.$$

Определение 5

$$L(M) = \{w \mid \hat{\delta}(q_0, w) \in F\}.$$

Язык **регулярный**, если распознаётся ДКА.

Принятие слова

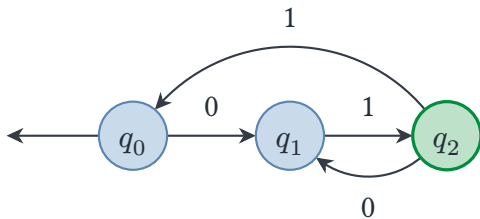
Пример

Для ДКА с $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1 \in F$: $\hat{\delta}(q_0, 01) = \delta(\hat{\delta}(q_0, 0), 1) = \delta(q_0, 1) = q_1 \in F$, значит $01 \in L(M)$.

Автомат принимает слово, если заканчивает в принимающем состоянии.

Регулярные языки — те, что допускают конечную память.

Заканчиваются на 01



Пример

Три состояния:

- ожидание;
- «последний 0»;
- «заканчивается на 01».

Прогон 1001: принимается.

Прогон 1000: отвергается.

Память автомата

Состояние хранит всю информацию о прочитанном префиксе, а переход её обновляет.

Трёх состояний хватает для слов любой длины.

Недетерминированные конечные автоматы

Определение НКА

Определение 6 (НКА)

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q).$$

Возвращает множество возможных состояний: $\delta(q_0, 0) = \{q_0, q_1\}$ — из одного состояния можно попасть сразу в несколько.

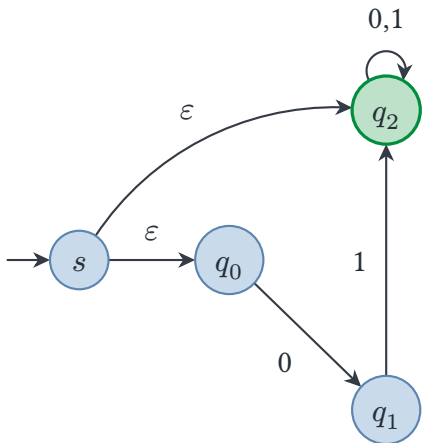
НКА принимает, если существует хотя бы один путь.

Никакого «угадывания»: лишь квантор существования.

Конструкция подмножеств убирает недетерминизм.

НКА с эпсилон-переходами

Язык: слова, содержащие подстроку 01, или пустое слово ϵ .



Пример

Три сценария:

- 011 принимается за счёт недетерминизма: по первому 0 идём в q_1 (ветвь «01»), а не по петле. Далее переход $q_1 \rightarrow q_2$ приводит в принимающее состояние, и машина остаётся в нём до конца слова.
- ϵ принимается за счёт ϵ -перехода: $s \rightarrow q_2$.
- 100 отвергается: ветвь q_1 умирает на втором 0, потому что перехода по 0 из q_1 нет. Машина, оставшаяся на петле q_0 , заканчивает в непринимающем состоянии.

Экспоненциальный разрыв

Язык «третий с конца символ равен 1»:

- НКА: при чтении 1 в любой позиции открывается ветвь, отсчитывающая до конца 3 символа, — всего 4 состояния.
- минимальный ДКА обязан различать все $2^3 = 8$ суффиксов длины 3.

Недетерминизм даёт экспоненциальный выигрыш в компактности: $k + 1$ состояний против 2^k .

Теорема Михилла–Нероуда

ДКА обязан хранить последние k символов, поэтому различает все 2^k суффиксов.

НКА проверяет все позиции параллельно: каждая ветвь отсчитывает оставшиеся символы.

Точный смысл 2^k даёт теорема Михилла–Нероуда: состояния минимального ДКА соответствуют классам строк, неразличимых по любому продолжению.

Число состояний — мера сложности языка, а не произвол конструктора.

Эквивалентность

Конструкция подмножеств

Теорема 1 (Эквивалентность ДКА и НКА)

Для любого НКА существует ДКА, распознающий тот же язык.

Доказательство

Состояния нового ДКА — подмножества состояний НКА.

Начальное состояние — ε -замыкание множества $\{q_0\}$: все состояния, достижимые из q_0 по ε -переходам.

Переход по символу a — ε -замыкание объединения множеств $\delta(q, a)$ по всем состояниям q текущего подмножества.

Принимающими объявляются подмножества, содержащие хотя бы одно принимающее состояние НКА.

Вычислительная сила

Конструкция подмножеств по любому НКА строит ДКА, распознающий тот же язык:

- недетерминизм не расширяет класс распознаваемых языков.
- цена — число состояний: до 2^k для НКА из k состояний (пример «третий с конца символ равен 1»: 4 против 8).

Недетерминизм не добавляет вычислительной силы.

Он добавляет компактность.

Теорема Клини

Теорема 2 (Клини)

Классы автоматных и регулярных языков совпадают.

Доказательство

В одну сторону регулярное выражение превращается в автомат, в другую — автомат в выражение.

От выражения к автомату. Разбираем выражение по структуре, собирая автомат из частей. Атомы: $\emptyset$ — автомат без пути из старта в конец, $\{\varepsilon\}$ — пустой переход, $\{a\}$ — переход по символу a . Конкатенация $L_1 L_2$ — пустой переход из конца автомата L_1 в старт автомата L_2 . Объединение $L_1 \vee L_2$ — новый старт с пустыми переходами в старты обоих автоматов.

Теорема Клини: звезда

Доказательство

Звезда L^* — возврат: пустые переходы из конца автомата L в её старт, а также в новый конец, принимающий в том числе пустую строку. Каждый шаг добавляет постоянное число состояний и переходов, поэтому выражение длины n даёт автомат из $O(n)$ состояний, распознающий ровно его язык.

Теорема Клини: от автомата к выражению

Доказательство

От автомата к выражению. Превратим НКА в обобщённый НКА с одним стартом и одним концом, где каждая дуга помечена регулярным выражением. Удаляем внутренние состояния по одному: если q — удаляемое состояние, а R_{iq} , R_{qq} и R_{qj} — выражения на дугах $i \rightarrow q$, $q \rightarrow q$ и $q \rightarrow j$, то дуга $i \rightarrow j$ получает выражение $R_{ij} \text{ mid } R_{iq}R_{qq}^*R_{qj}$. Удалив все внутренние состояния, получаем единственную дугу от нового старта к новому концу, и её метка — регулярное выражение языка исходного автомата.

Регулярный язык — язык, который собирается из атомов тремя операциями:

- атомы: $\emptyset$, язык $\{\varepsilon\}$ из одного пустого слова, язык $\{a\}$ из одного символа $a \in \Sigma$.
- операции: конкатенация L_1L_2 , объединение $L_1 \mid L_2$, звезда L^* (повторение слова).

Двусторонняя связь

Связь с автоматами двусторонняя:

- выражение $\rightarrow$ автомат: конструкция Томпсона, автоматы компонуются по операциям выражения.
- автомат $\rightarrow$ выражение: алгоритм Клини, состояния превращаются в выражения.

Пример

$(0 \mid 1)^*01$ — слова, заканчивающиеся на 01.

Звезда Клини — сжатая запись цикла. Грег и лексеры — теорема Клини в действии.

Нерегулярность

Лемма о накачке

Теорема 3 (Лемма о накачке)

Для регулярного L существует p , такое что любое слово $w \in L$ длины $|w| \geq p$ разбивается как $x y z$:

- $|y| > 0$;
- $|xy| \leq p$.
- $x y^i z \in L$ для всех $i \geq 0$.

Доказательство

При чтении длинного слова состояние повторяется (принцип Дирихле).

Повтор — цикл, который можно накачать.

Лемма даёт необходимое условие: с её помощью регулярность опровергают.

Пример нерегулярного языка

Пример

$L = \{0^n 1^n\}$: возьмём $w = 0^p 1^p$.

xu целиком в первых p символах: $y = 0^k$.

Накачанное $0^{p+k} 1^p$ не в языке — противоречие.

Конечная память не отличает «поровну».

Чтобы отличить «поровну» от «не поровну», нужен счёт, а конечная память его не даёт.

Что мы поняли?

- Язык — множество слов $L \subseteq \Sigma^*$. Распознать его значит решить для каждого слова, лежит ли оно в L .
- ДКА хранит всю память в номере текущего состояния: регулярные языки допускают конечную память.
- ДКА, НКА и регулярные выражения задают один и тот же класс языков — связывает их конструкция подмножеств и теорема Клини.
- Лемма о накачке — способ доказать нерегулярность: $\{0^n 1^n\}$ требует счёта, которого конечная память не даёт.

Вопросы для самопроверки

- Что такое формальный язык?
- Определите ДКА и язык, который он распознаёт.
- Почему НКА не «угадывает»?
- Как конструкция подмножеств убирает недетерминизм?
- Сформулируйте теорему Клини.
- Сформулируйте лемму о накачке.
- Докажите, что $\{0^n 1^n\}$ нерегулярен.
- Зачем нужны регулярные выражения в программировании?